The HHL Algorithm

Solving linear systems $A\vec{x} = \vec{b}$ on a quantum computer — explained step by step

CUDA-Q Quantum Algorithm Benchmarking Suite

From linear algebra to a working CUDA-Q implementation

All quantum results in these slides were computed with NVIDIA CUDA-Q (CPU simulator `qpp-cpu`) and checked against NumPy. No quantum hardware was used.

What you will learn today

1. **Linear algebra** for HHL: linear systems, eigenvalues, Hermitian and sparse matrices, condition number
2. **Quantum tools**: amplitude encoding, controlled gates, measurement, QFT, **phase estimation**, Hamiltonian simulation
3. The **HHL algorithm**, derived step by step: registers, eigenvalue inversion, uncomputation, post-selection
4. Worked 2×2 and 4×4 examples
5. **Complexity**: where the speed-up comes from and where it disappears
6. The **CUDA-Q implementation** in `HHL.ipynb`, with real results
7. Noise, limitations, applications and **advanced algorithms**

You only need

- matrices and vectors
- complex numbers $e^{i\phi}$
- basic qubits and gates (the QFT lecture is a good preparation)

Everything else is explained on the way.

The big picture in one slide

- Classical task: given A $(d \times d)$ and $\vec{b}$, find $\vec{x}$ with $A\vec{x} = \vec{b}$.
- HHL (Harrow, Hassidim, Lloyd, 2009) prepares the **quantum state**

$$|x\rangle = \frac{A^{-1}|b\rangle}{\|A^{-1}|b\rangle\|}$$

  using only $\log_2 d$ qubits for the data.
- Idea: write $|b\rangle$ in the eigenbasis of A and **divide every component by its eigenvalue**.
- Cost $\tilde{O}(\log(d)\, s^2 \kappa^2/\epsilon)$ — but only under strong **assumptions**, and the output is a state, **not a list of numbers**.

Keep in mind all lecture long

HHL is **not** a faster `numpy.linalg.solve`. It is a subroutine that produces $|x\rangle$ so that a *property* of $\vec{x}$ (an expectation value, an overlap) can be estimated.

Vectors, matrices and linear systems

A **linear system** of d equations in d unknowns:

$$\begin{aligned} 2x_1 + x_2 &= 1 \\ x_1 + 3x_2 &= 2 \end{aligned} \iff \underbrace{\begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}}_{A} \underbrace{\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}}_{\vec{x}} = \underbrace{\begin{pmatrix} 1 \\ 2 \end{pmatrix}}_{\vec{b}}$$

- Unique solution $\iff \det A \neq 0$ (A invertible): $\vec{x} = A^{-1}\vec{b}$.
- 2×2: $A^{-1} = \frac{1}{ad-bc}\begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$.

Worked example

$\det A = 6 - 1 = 5$,

$$\vec{x} = \frac{1}{5}\begin{pmatrix} 3 & -1 \\ -1 & 2 \end{pmatrix}\begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 0.2 \\ 0.6 \end{pmatrix}.$$

Check: $2(0.2) + 0.6 = 1$,
$0.2 + 3(0.6) = 2$. ✓

Linear systems appear in circuit simulation, finite elements, fluid dynamics, least-squares fitting, machine learning, network flow ...

Classical methods for $A\vec{x} = \vec{b}$

Method	Cost	Comment
Gaussian elimination / LU	$O(d^3)$	direct, dense, factorisation reusable
Cholesky (A positive definite)	$\approx \frac{1}{3}d^3$	half the work of LU
Conjugate gradient (A PD, s-sparse)	$O(d\, s\sqrt{\kappa}\log\frac{1}{\epsilon})$	iterative, matrix–vector products only
Krylov methods (GMRES, ...)	$O(d\, s \cdot \text{iterations})$	general sparse matrices

- Every classical method must at least **read** $\vec{b}$ and **write** $\vec{x}$: cost $\geq d$.
- HHL's exponential improvement in d is only possible because it does **neither**: $|b\rangle$ is prepared by a circuit and $|x\rangle$ is never written out.
- The reference in the notebook is `np.linalg.solve` (LAPACK, LU with pivoting).

Matrix properties that matter for HHL

- **Conjugate transpose**: $A^\dagger = \bar{A}^T$.
- **Hermitian**: $A = A^\dagger$ (real symmetric is a special case).
- **Unitary**: $U^\dagger U = I$ — preserves lengths; all quantum gates are unitary.
- **Invertible**: $\det A \neq 0$, no zero eigenvalue.
- **Positive definite**: Hermitian with all eigenvalues > 0.
- **Sparse**: at most $s \ll d$ non-zeros per row.

Quick checks

$\left(\begin{smallmatrix} 0 & 1 \\ 1 & 0 \end{smallmatrix}\right)$	Hermitian *and* unitary
$\left(\begin{smallmatrix} 1 & i \\ -i & 2 \end{smallmatrix}\right)$	Hermitian, $\det = 1$
$\left(\begin{smallmatrix} 1 & 2 \\ 2 & 4 \end{smallmatrix}\right)$	Hermitian, singular
$\left(\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right)$	not Hermitian

HHL needs a Hermitian, invertible A. Other matrices are **embedded** (next slides).

Eigenvalues and eigenvectors

$$A\,|u_j\rangle = \lambda_j\,|u_j\rangle \qquad \det(A - \lambda I) = 0$$

For a **Hermitian** matrix (spectral theorem):

- all λ_j are **real**;
- the $|u_j\rangle$ form an **orthonormal basis**;
- $A = \sum_j \lambda_j\,|u_j\rangle\langle u_j| = V\Lambda V^\dagger$.

Proof of realness:
$\lambda\,\langle u|u\rangle = \langle u|A|u\rangle = \langle u|A^\dagger|u\rangle^* = \bar{\lambda}\,\langle u|u\rangle$.

Example

$A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$: $\lambda^2 - 4\lambda + 3 = 0$

- $\lambda_1 = 3$, $|u_1\rangle = \frac{1}{\sqrt{2}}(1, 1)$
- $\lambda_2 = 1$, $|u_2\rangle = \frac{1}{\sqrt{2}}(1, -1)$

Check: trace $4 = 3 + 1$, determinant $3 = 3 \cdot 1$.

Functions of a matrix — and the inverse

With $A = \sum_j \lambda_j |u_j\rangle\langle u_j|$, any function acts on the eigenvalues:

$$f(A) = \sum_j f(\lambda_j) |u_j\rangle\langle u_j| .$$

Two functions are central for HHL:

- $f(\lambda) = e^{i\lambda t}$: $U = e^{iAt}$ is **unitary** (a quantum gate!).
- $f(\lambda) = 1/\lambda$: $A^{-1} = \sum_j \lambda_j^{-1} |u_j\rangle\langle u_j|$.

HHL uses the first to **measure** the eigenvalues and then applies the second.

Solving in the eigenbasis

$$\vec{b} = \sum_j \beta_j |u_j\rangle , \quad \beta_j = \langle u_j | b \rangle$$

$$\boxed{\vec{x} = A^{-1}\vec{b} = \sum_j \frac{\beta_j}{\lambda_j} |u_j\rangle}$$

Example above with $\vec{b} = (1, 0)$:
$\beta_1 = \beta_2 = \frac{1}{\sqrt{2}}$,
$\vec{x} = \frac{1}{6}(1,1) + \frac{1}{2}(1,-1) = (\frac{2}{3}, -\frac{1}{3})$.

Sparse matrices

- **Sparsity** s = maximum number of non-zero entries in any row.
- Example: 1-D Laplacian (heat equation, springs, circuits)

$$L = \begin{pmatrix} 2 & -1 & & \\ -1 & 2 & -1 & \\ & \ddots & \ddots & \ddots \\ & & -1 & 2 \end{pmatrix}, \quad s = 3$$

(8×8: 22 non-zeros instead of 64).

- Sparse $A \Rightarrow e^{iAt}$ can be simulated with $\mathrm{poly}(s, \log d)$ gates.

Why HHL wants sparsity

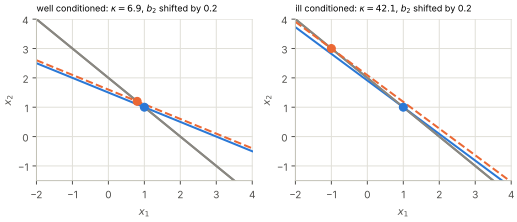
HHL needs controlled e^{iAt}. For an s-sparse matrix given by an *oracle* ("which entries are non-zero in row i, and what are they?"), Hamiltonian simulation costs about s^2 (original HHL) or s (modern methods) — independent of d up to $\log d$.

Dense matrices need other access models (block encodings / QRAM).

The condition number κ

$$\kappa(A) = \|A\|\,\|A^{-1}\| = \frac{\sigma_{\max}}{\sigma_{\min}} \overset{\text{Hermitian}}{=} \frac{|\lambda|_{\max}}{|\lambda|_{\min}}$$

- Sensitivity: $\dfrac{\|\delta\vec{x}\|}{\|\vec{x}\|} \le \kappa \dfrac{\|\delta\vec{b}\|}{\|\vec{b}\|}$
- $\kappa = 1$: unitary (perfect); $\kappa = \infty$: singular.
- Laplacian: $\kappa \approx 4(n+1)^2/\pi^2$ grows with the grid.
- For HHL: run time $\propto \kappa^2$ (original), success probability $\sim 1/\kappa^2$.



Two lines meeting at a shallow angle: a small change of $\vec{b}$ (dashed) moves the solution far.

Two preparation tricks: Hermitian embedding and padding

Non-Hermitian A? Solve the doubled system

$$\begin{pmatrix} 0 & A \\ A^\dagger & 0 \end{pmatrix} \begin{pmatrix} \vec{y} \\ \vec{z} \end{pmatrix} = \begin{pmatrix} \vec{b} \\ \vec{0} \end{pmatrix} \Rightarrow A\vec{z} = \vec{b},\ \vec{y} = \vec{0}.$$

- The embedded matrix is Hermitian with eigenvalues $\pm\sigma_i(A)$.
- Cost: **one extra qubit**; the solution is the second half.

Example: $A = \left(\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right)$: eigenvalues $\pm(\sqrt{2} \pm 1)$, $\kappa = 3 + 2\sqrt{2}$.

Dimension not a power of 2? Pad with an identity block:

$$\begin{pmatrix} A & 0 \\ 0 & I \end{pmatrix} \begin{pmatrix} \vec{x} \\ \vec{0} \end{pmatrix} = \begin{pmatrix} \vec{b} \\ \vec{0} \end{pmatrix}.$$

In `HHL.ipynb`

Both are automatic: a non-Hermitian 5×5 matrix becomes 10×10, then 16×16: $n_b = 4$ system qubits.

Qubits and amplitude encoding

- n qubits hold 2^n complex amplitudes: $|\psi\rangle = \sum_{i=0}^{2^n-1} a_i |i\rangle$, $\sum |a_i|^2 = 1$.
- **Amplitude encoding**: a vector $\vec{b} \in \mathbb{C}^d$ becomes

$$|b\rangle = \frac{1}{\|\vec{b}\|} \sum_i b_i |i\rangle$$

  on $n_b = \log_2 d$ qubits.
- The **norm is lost**: it must be remembered classically.
- Measuring gives i with probability $|b_i|^2 / \|\vec{b}\|^2$.

Example

$\vec{b} = (3, 0, 4, 0)$, $\|\vec{b}\| = 5$:

$$|b\rangle = 0.6\,|00\rangle + 0.8\,|10\rangle$$

$P(00) = 0.36$, $P(10) = 0.64$.

Convention here and in the notebook: **qubit 0 is written first** and is the most significant bit.

Assumption of HHL

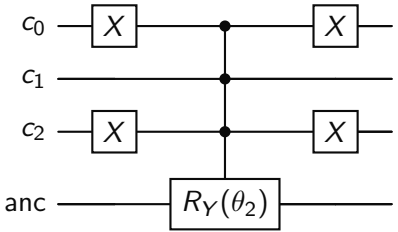
$|b\rangle$ can be prepared efficiently. A general $\vec{b}$ needs $O(d)$ gates.

Gates, rotations and controlled gates

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad R_Y(\theta) = \begin{pmatrix} \cos\frac{\theta}{2} & -\sin\frac{\theta}{2} \\ \sin\frac{\theta}{2} & \cos\frac{\theta}{2} \end{pmatrix}$$

$$R_Y(\theta)\,|0\rangle = \cos\tfrac{\theta}{2}\,|0\rangle + \sin\tfrac{\theta}{2}\,|1\rangle$$

- With $\theta = 2\arcsin c$ the amplitude of $|1\rangle$ is exactly c — this is how HHL writes $1/\lambda$ into an amplitude.
- **Controlled-U**: $|0\rangle\,|\psi\rangle \mapsto |0\rangle\,|\psi\rangle$, $|1\rangle\,|\psi\rangle \mapsto |1\rangle\,U\,|\psi\rangle$.
- **Multi-controlled** gates fire only on $|1\ldots1\rangle$; X gates around them select any other pattern.



Rotation applied only when the clock reads $c_0c_1c_2 = 010$ ($k = 2$).

Measurement, entanglement and post-selection

- Born rule: outcome k with probability $|a_k|^2$; the state collapses — **irreversible**.
- **Entangled** states cannot be written as products, e.g. $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Measuring one part changes the other.
- **Post-selection**: run the circuit, measure a flag qubit and keep only the runs with the wanted outcome:

$$\alpha\,|0\rangle\,|\psi_0\rangle + \beta\,|1\rangle\,|\psi_1\rangle \xrightarrow{\text{flag}=1} |\psi_1\rangle$$

with probability $P = |\beta|^2$.

Why it matters for HHL

- $1/\lambda$ is not unitary, so HHL cannot apply A^{-1} directly.
- It applies a unitary that puts $CA^{-1}\,|b\rangle$ into the ancilla-1 branch and **post-selects** it.
- Price: success probability $P_{\text{success}} = C^2\|A^{-1}\,|b\rangle\|^2 < 1$.

Recap: the quantum Fourier transform

$$\mathrm{QFT}\,|k\rangle = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} e^{2\pi i\, jk/N}\,|j\rangle\,, \quad N = 2^n$$

- A **phase gradient** (phase advancing by $2\pi k/N$ per step) is $\mathrm{QFT}\,|k\rangle$.
- So the **inverse QFT** turns a phase gradient into the integer k.
- Circuit: n Hadamards, $n(n-1)/2$ controlled phases, $\lfloor n/2 \rfloor$ SWAPs — $O(n^2)$ gates.

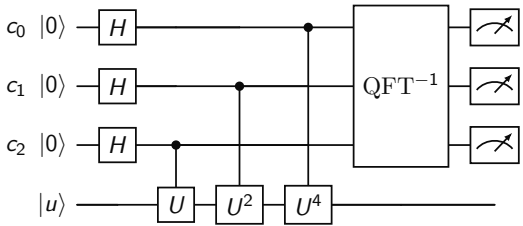
Example ($n = 2$)

$\frac{1}{2}(1, i, -1, -i)$ advances by $\pi/2 = 2\pi \cdot 1/4$ per step: QFT^{-1} gives $|01\rangle$.

$\frac{1}{2}(1, -i, -1, i)$ advances by $3 \cdot 2\pi/4$: gives $|11\rangle$.

For the full derivation see the QFT lecture (`presentations/QFT_Lecture.pdf`).

Quantum phase estimation (QPE)



$U|u\rangle = e^{2\pi i\varphi}|u\rangle$. **Phase kickback**: controlled-U^{2^j} on $|+\rangle|u\rangle$ gives $\frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i\varphi 2^j}|1\rangle)|u\rangle$. All together:

$$\frac{1}{\sqrt{2^{n_c}}}\sum_k e^{2\pi i\varphi k}|k\rangle\,|u\rangle \xrightarrow{\mathrm{QFT}^{-1}} |\varphi\, 2^{n_c}\rangle\,|u\rangle$$

if $\varphi 2^{n_c}$ is an integer. Otherwise the outcome is the nearest integer with probability $\geq 4/\pi^2$:

$$P(k) = \frac{\sin^2(\pi 2^{n_c}\delta)}{2^{2n_c}\sin^2(\pi\delta)},\ \delta = \varphi - \tfrac{k}{2^{n_c}}.$$

QPE examples

Exact phase

$\varphi = 0.375 = 0.011_2$, $n_c = 3$: clock $\to |011\rangle$ with probability 1.

$\varphi = \frac{1}{4}$, $n_c = 2$: amplitudes $\frac{1}{2}(1, i, -1, -i)$ $\to |01\rangle$.

Non-representable phase

$\varphi = \frac{1}{3}$, $n_c = 3$: $8\varphi = 2.67$.
$P(3) = 0.688$, $P(2) = 0.175$, rest spread out.

QPE in HHL

$U = e^{iAt}$ has eigenphases $\varphi_j = \lambda_j t/2\pi$, so QPE on $|u_j\rangle$ writes $k_j = \lambda_j t\, 2^{n_c}/2\pi$ into the clock. The **evolution time** t sets the scale; HHL is **exact** when every k_j is an integer and **approximate** otherwise. $|b\rangle$ is not an eigenvector — but QPE is linear, so each eigencomponent gets its own clock value.

Hamiltonian simulation: implementing e^{iAt}

QPE needs **controlled** $U^{2^j} = e^{iAt2^j}$ for $j = 0, \ldots, n_c - 1$.

- **Exact** (small systems, simulators): compute e^{iAt} classically and use it as a gate — what `HHL.ipynb` does.
- **Trotterisation**: $A = \sum_\ell A_\ell$ (easy pieces, e.g. Pauli strings):

$$e^{iAt} \approx \Big(\prod_\ell e^{iA_\ell t/r} \Big)^r$$

  with error $\approx \frac{t^2}{2r} \sum_{\ell<m} \|[A_\ell, A_m]\|$, so $r = O(t^2/\epsilon)$ steps.
- **Sparse-oracle methods**, quantum walks, qubitisation and QSVT: near-optimal $O(st + \log(1/\epsilon))$.

Hidden cost

Total evolution time in QPE is $t(2^{n_c} - 1) \sim \kappa/\epsilon$. With $\kappa = 100$, $\epsilon = 0.01$ this is $\sim 10^4$ units of t — deep circuits.

A Hermitian A is required: only then is e^{iAt} unitary.

Quantum linear algebra: the idea

- A quantum computer can apply some functions of a matrix to a state **without writing the matrix down**: e^{iAt}, polynomials of A, A^{-1} (approximately).
- Ingredients: an efficient **access model** for A (sparse oracle, block encoding), **phase estimation** or polynomial transforms, **amplitude amplification**.
- Output: a quantum state — to be used by another quantum routine or measured for a few numbers.

Family tree

- HHL (2009): QPE + controlled rotation
- Ambainis (2012): variable-time amplification, $\tilde{O}(\kappa)$
- Childs–Kothari–Somma (2017): LCU, $\mathrm{polylog}(1/\epsilon)$
- QSVT (2019): unified polynomial framework
- Costa et al. (2022): optimal $O(\kappa \log \frac{1}{\epsilon})$

Problem formulation and assumptions

Input: Hermitian A ($d = 2^{n_b}$), vector $\vec{b}$.
Output: the state

$$|x\rangle = \frac{A^{-1}|b\rangle}{\|A^{-1}|b\rangle\|} = \frac{\sum_j (\beta_j/\lambda_j)|u_j\rangle}{\sqrt{\sum_j |\beta_j|^2/\lambda_j^2}}.$$

Normalise A so that its eigenvalues lie in a known range, e.g. $|\lambda| \in [1/\kappa, 1]$, and choose t accordingly.

The assumptions (and why)

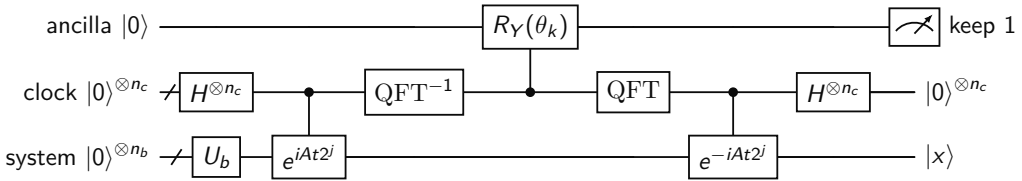
1. A Hermitian — e^{iAt} must be unitary.
2. A sparse / efficiently simulable — cheap controlled e^{iAt}.
3. κ small — precision and success probability.
4. $|b\rangle$ efficiently preparable — otherwise input costs $O(d)$.
5. Only a *property* of $|x\rangle$ is needed — read-out costs $O(d)$.
6. Eigenvalues resolvable by n_c clock qubits.

The algorithm in five steps

	Step	Operation	State (clock, system, ancilla)
0	prepare	load $\lvert b\rangle$	$\lvert 0\rangle \lvert b\rangle \lvert 0\rangle = \sum_j \beta_j \lvert 0\rangle \lvert u_j\rangle \lvert 0\rangle$
1	estimate	QPE with $U = e^{iAt}$	$\sum_j \beta_j \lvert k_j\rangle \lvert u_j\rangle \lvert 0\rangle$
2	invert	controlled $R_Y(\theta_k)$	$\sum_j \beta_j \lvert k_j\rangle \lvert u_j\rangle \left(\sqrt{1 - \frac{C^2}{\lambda_j^2}} \lvert 0\rangle + \frac{C}{\lambda_j} \lvert 1\rangle\right)$
3	uncompute	QPE^{-1}	$\sum_j \beta_j \lvert 0\rangle \lvert u_j\rangle \left(\sqrt{\cdot} \lvert 0\rangle + \frac{C}{\lambda_j} \lvert 1\rangle\right)$
4	measure	ancilla $= 1$	$\lvert 0\rangle \otimes \frac{\sum_j (\beta_j/\lambda_j) \lvert u_j\rangle}{\lVert\cdot\rVert} = \lvert 0\rangle \lvert x\rangle$

$k_j = \lambda_j t \, 2^{n_c}/2\pi$ (assumed exact here), $C \leq \min_j |\lambda_j|$, success probability $P = \sum_j |\beta_j|^2 C^2/\lambda_j^2$.

HHL circuit architecture



Clock (n_c qubits): binary eigenvalue estimates k. Precision $2\pi/(2^{n_c}t)$.

System ($n_b = \log_2 d$): starts as $|b\rangle$ (U_b = state preparation), ends as $|x\rangle$.

Ancilla (1 qubit): carries C/λ; measuring 1 = success.

Qubits: $n_c + n_b + 1$. Notebook default: $4 + 2 + 1 = 7$. The middle part ($\mathrm{QFT}^\dagger$, rotation, QFT, inverse powers, H) is drawn compactly.

Step 0: state preparation

- Load $|b\rangle = \vec{b}/\|\vec{b}\|$ into the system register; keep $\|\vec{b}\|$ classically.
- Expand (only in our heads, never computed):

$$|b\rangle = \sum_j \beta_j |u_j\rangle\,, \quad \sum_j |\beta_j|^2 = 1.$$

- Structured vectors are cheap: $(1,-1,1,-1)/2 = |+\rangle\,|-\rangle$ needs $H \otimes H$ and one Z.
- General vectors: $O(d)$ rotations (Grover–Rudolph / Möttönen), or QRAM in $O(\log d)$ depth with $O(d)$ hardware.

Notebook

`HHL.ipynb` loads the whole initial state $|0\ldots0\rangle_{\text{clock}}\,|b\rangle\,|0\rangle$ at once with `kernel.qalloc(vector)` — exact amplitude initialisation, available on simulators only.

Step 1: phase estimation of the eigenvalues

For one eigencomponent:
$e^{iAt}\,|u_j\rangle = e^{i\lambda_j t}\,|u_j\rangle = e^{2\pi i\varphi_j}\,|u_j\rangle$ with $\varphi_j = \frac{\lambda_j t}{2\pi}$.

1. Hadamards: $\frac{1}{\sqrt{2^{n_c}}}\sum_m |m\rangle\,|u_j\rangle$
2. controlled e^{iAt2^ℓ} by the clock bit of weight 2^ℓ: $\frac{1}{\sqrt{2^{n_c}}}\sum_m e^{i\lambda_j t m}\,|m\rangle\,|u_j\rangle$
3. QFT^{-1}: $|k_j\rangle\,|u_j\rangle$, $k_j = \dfrac{\lambda_j t\, 2^{n_c}}{2\pi}$

By linearity, for $|b\rangle = \sum_j \beta_j\,|u_j\rangle$:

$$|0\rangle\,|b\rangle \longmapsto \sum_j \beta_j\,|k_j\rangle\,|u_j\rangle\,.$$

Clock and system are now **entangled**.

Choosing t

- $|\lambda|t < \pi$ for all λ: no aliasing.
- Notebook "auto": $t = \frac{2\pi}{2^{n_c}|\lambda|_{\min}}$ maps $|\lambda|_{\min}$ to $k = 1$ (if $\kappa \leq 2^{n_c-1} - 1$).
- Default: $n_c = 4$, $\lambda_{\min} = 1 \Rightarrow$ $t = \pi/8$ and $k_j = \lambda_j$.

Negative eigenvalues: two's complement

- Phases are periodic: $e^{i\lambda t}$ with $\lambda < 0$ equals $e^{i(\lambda t + 2\pi)}$.
- So $\lambda < 0$ lands on clock value $k = 2^{n_c} + \lambda t 2^{n_c}/2\pi$, near the top.
- Interpretation: $k \geq 2^{n_c - 1}$ means the **signed** value $k - 2^{n_c}$.
- Then $\tilde{\lambda}_k = kC$ (signed) and the rotation angle $2\arcsin(C/\tilde{\lambda}_k)$ is **negative**: the sign of $1/\lambda$ survives.

Example ($n_c = 4$, $t = \pi/8$, $C = 1$)

$\lambda = -2$: $e^{-2it} = e^{2\pi i \cdot 14/16}$, so
$k = 14 = 1110_2$.
Signed: $14 - 16 = -2$,
$\theta = 2\arcsin(-\frac{1}{2}) = -\pi/3$.

Needed for embedded non-Hermitian matrices, whose eigenvalues come in $\pm$ pairs. Positive range: $k = 1, \ldots, 2^{n_c - 1} - 1$.

Step 2: eigenvalue inversion by a controlled rotation

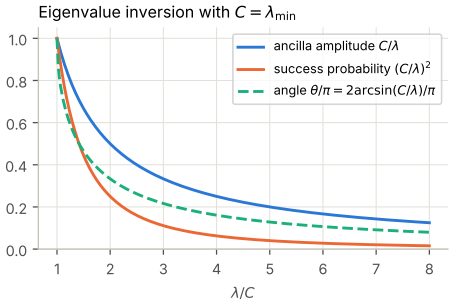
For every clock value k ($\tilde{\lambda}_k = kC$) rotate the ancilla:

$$|k\rangle\,|0\rangle_a \mapsto |k\rangle\left(\sqrt{1-\frac{C^2}{\tilde{\lambda}_k^2}}\,|0\rangle + \frac{C}{\tilde{\lambda}_k}\,|1\rangle\right)$$

i.e. $R_Y(\theta_k)$ with $\theta_k = 2\arcsin(C/\tilde{\lambda}_k)$.
Applied to the state of step 1:

$$\sum_j \beta_j\,|k_j\rangle\,|u_j\rangle\left(\sqrt{\cdot}\,|0\rangle + \frac{C}{\lambda_j}\,|1\rangle\right).$$

The **ancilla-1 branch carries** $\beta_j C/\lambda_j$ — exactly the coefficients of $C\,A^{-1}\,|b\rangle$.



The constant C

Need $|C/\tilde{\lambda}| \le 1 \Rightarrow C \le \min|\lambda|$. Larger $C \Rightarrow$ higher $P_{\text{success}} \propto C^2$. Notebook: $C = \frac{2\pi}{2^{n_c} t}$ (clock value 1); default $C = 1$.

How the rotation is built (notebook version)

- "Look-up table": one multi-controlled $R_Y(\theta_k)$ per clock value $k = 1, \ldots, 2^{n_c} - 1$.
- X gates on the clock qubits whose bit of k is 0 before and after ("polarity").
- $k = 0$ gets no rotation (it would mean $\lambda = 0$).
- Default $n_c = 4$: 15 rotations, each controlled by 4 qubits, and 56 X gates.

Scaling

$2^{n_c} - 1$ rotations is exponential in n_c. Scalable versions compute $1/\lambda$ with reversible arithmetic, or approximate $1/x$ by a polynomial (QSVT).

Angles in the default run

k	bits	$\tilde{\lambda}_k$	θ_k
1	0001	1	π
2	0010	2	$\pi/3$
3	0011	3	$2 \arcsin \frac{1}{3} = 0.680$
4	0100	4	$2 \arcsin \frac{1}{4} = 0.505$
14	1110	-2	$-\pi/3$

$\theta_1 = \pi$ flips the ancilla fully: amplitude $C/\lambda = 1$.

Step 3: uncomputation

Apply phase estimation **in reverse** (QFT, inverse controlled powers e^{-iAt2^ℓ} in reverse order, Hadamards):

$$|k_j\rangle\,|u_j\rangle \longmapsto |0\rangle\,|u_j\rangle\,.$$

Result:

$$|0\rangle_{\text{clock}} \sum_j \beta_j\,|u_j\rangle \left(\sqrt{1-\frac{C^2}{\lambda_j^2}}\,|0\rangle + \frac{C}{\lambda_j}\,|1\rangle\right).$$

Why not just measure the clock?

Measuring gives one k_j and collapses the system to a *single* eigenvector $|u_j\rangle$. Not uncomputing at all leaves the clock as a "label": the system is in a *mixture*. Only coherent uncomputation lets the components interfere into $|x\rangle$.

Rule: undo = reverse order and † every gate.

Step 4: measurement and post-selection

Measure the ancilla. Outcome 1 with probability

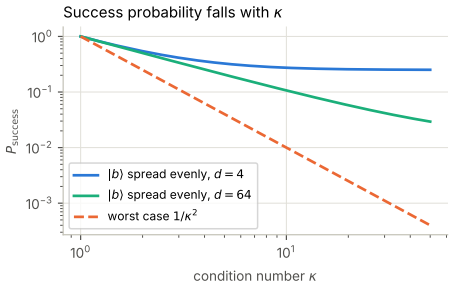
$$P_{\text{success}} = \sum_j |\beta_j|^2 \frac{C^2}{\lambda_j^2} = C^2 \|A^{-1}|b\rangle\|^2,$$

leaving

$$\frac{\sum_j (\beta_j/\lambda_j)|u_j\rangle}{\sqrt{\sum_j |\beta_j|^2/\lambda_j^2}} = |x\rangle\,.$$

Outcome 0: discard and repeat.

- Worst case $P \sim C^2/\lambda_{\text{max}}^2 \sim 1/\kappa^2$: about κ^2 repetitions.
- **Amplitude amplification**: $O(\kappa)$ instead.



Success probability with $C = \lambda_{\text{min}}$ (formula).

Extracting information from $|x\rangle$

- **Sampling** the system: outcome i with probability $|x_i|^2/\|\vec{x}\|^2$ — magnitudes only.
- All d values to precision ϵ: $\sim d/\epsilon^2$ runs (tomography), plus other bases for signs and phases. **This cancels the speed-up.**
- **Efficient outputs**: $\langle x|M|x\rangle$ for a measurable M ($O(1/\epsilon^2)$ runs), overlaps $|\langle y|x\rangle|^2$ (swap test), the weight of a region of $\vec{x}$.
- **Norm**: $\|\vec{x}\| = \frac{\|\vec{b}\|}{C}\sqrt{P_{\text{success}}}$ — from the success rate.

On a simulator

The full state vector is available, so the notebook reads the post-selected amplitudes $\vec{a}$ directly and recovers

$$\vec{x} = \frac{\|\vec{b}\|}{C}\,\vec{a}$$

including signs and norm. On hardware this is **impossible**; only samples exist.

HHL as pseudocode

```
input : Hermitian A (d = 2^n_b), vector b, clock qubits n_c, time t
C = 2*pi / (2^n_c * t) # smallest representable |eigenvalue|
repeat
    prepare |0...0>_clock |b>_system |0>_anc
    H on every clock qubit
    for l in 0..n_c-1: controlled-exp(i*A*t*2^l), control = clock bit of weight 2^l
    inverse QFT on clock # clock now holds k_j = lambda_j t 2^n_c / 2pi
    for k in 1..2^n_c-1: # eigenvalue inversion
        lam_k = signed(k) * C
        multi-controlled RY(2*asin(C/lam_k)) on anc, active when clock == k
    QFT on clock; inverse controlled powers; H on clock # uncompute
    measure anc
until anc == 1 # expected ~ 1/P_success repetitions
return system register (= |x>, proportional to A^-1 b)
```

Worked 2×2 example: setting up

$$A = \begin{pmatrix} \frac{3}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{3}{2} \end{pmatrix}, \qquad \vec{b} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}.$$

- $\lambda_1 = 1$, $|u_1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$
- $\lambda_2 = 2$, $|u_2\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$
- $\kappa = 2$; $|b\rangle = |0\rangle = \frac{1}{\sqrt{2}}(|u_1\rangle + |u_2\rangle)$

Classical answer: $\vec{x} = A^{-1}\vec{b} = (\frac{3}{4}, -\frac{1}{4})$.

Parameters

- $n_b = 1$, $n_c = 2$, one ancilla: 4 qubits
- $t = \pi/2$: $k = \lambda t\, 2^{n_c}/2\pi = \lambda$
- $C = 2\pi/(4 \cdot \pi/2) = 1$
- both eigenvalues positive: read k unsigned here

Worked 2×2 example: the four steps

1. **After QPE**: $|01\rangle$ for $\lambda_1 = 1$, $|10\rangle$ for $\lambda_2 = 2$:

$$\tfrac{1}{\sqrt{2}}\big(\,|01\rangle\,|u_1\rangle + |10\rangle\,|u_2\rangle\,\big)\,|0\rangle_a$$

2. **Rotation**: $k = 1$: $\theta = 2\arcsin 1 = \pi$ (amplitude 1); $k = 2$: $\theta = \pi/3$ (amplitude $\tfrac{1}{2}$):

$$\tfrac{1}{\sqrt{2}}\Big(\,|01\rangle\,|u_1\rangle\,|1\rangle + |10\rangle\,|u_2\rangle\,\big(\tfrac{\sqrt{3}}{2}\,|0\rangle + \tfrac{1}{2}\,|1\rangle\,\big)\Big)$$

3. **Uncompute**: clock $\to |00\rangle$. Ancilla-1 branch:
$\tfrac{1}{\sqrt{2}}\big(\,|u_1\rangle + \tfrac{1}{2}\,|u_2\rangle\,\big) = \tfrac{1}{2}[(|0\rangle - |1\rangle) + \tfrac{1}{2}(|0\rangle + |1\rangle)] = \tfrac{3}{4}\,|0\rangle - \tfrac{1}{4}\,|1\rangle$

4. **Measure**: $P = (\tfrac{3}{4})^2 + (\tfrac{1}{4})^2 = \tfrac{5}{8}$; state $\tfrac{1}{\sqrt{10}}(3\,|0\rangle - |1\rangle)$.

Check

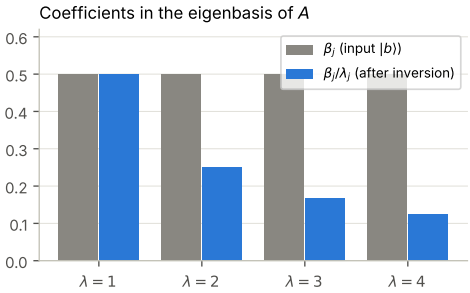
With $\|\vec{b}\| = C = 1$ the branch *is* $\vec{x} = (\tfrac{3}{4}, -\tfrac{1}{4})$; $P = \|\vec{x}\|^2 = \tfrac{10}{16}$. Verified with CUDA-Q in `verify_answers.py`.

Worked 4×4 example (the notebook default)

$$A = \begin{pmatrix} 2.5 & -0.5 & -1 & 0 \\ -0.5 & 2.5 & 0 & -1 \\ -1 & 0 & 2.5 & -0.5 \\ 0 & -1 & -0.5 & 2.5 \end{pmatrix} = H_4 \,\mathrm{diag}(1,2,3,4)\, H_4$$

with $H_4 = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & -1 & 1 & -1 \\ 1 & 1 & -1 & -1 \\ 1 & -1 & -1 & 1 \end{pmatrix}$, $H_4^2 = I$.

- Eigenvalues $1, 2, 3, 4$, $\kappa = 4$; dense ($s = 3$).
- $\vec{b} = (1,0,0,0)$: $\beta_j = \frac{1}{2}$ for all j.
- $n_c = 4$, $t = \pi/8$, $C = 1$: clock values $k = 1, 2, 3, 4$ exactly.



Inversion shrinks large-eigenvalue components.

Worked 4×4 example: the numbers

$$A^{-1}\vec{b} = H_4 \Lambda^{-1} H_4 \vec{b} = H_4 \cdot \tfrac{1}{2}\left(1, \tfrac{1}{2}, \tfrac{1}{3}, \tfrac{1}{4}\right)$$
$$= \tfrac{1}{48}(25,\ 7,\ 11,\ 5)$$
$$= (0.520833,\ 0.145833,\ 0.229167,\ 0.104167)$$

$$P_{\text{success}} = \tfrac{1}{4}\left(1 + \tfrac{1}{4} + \tfrac{1}{9} + \tfrac{1}{16}\right) = \tfrac{205}{576} = 0.355903$$

$$\|\vec{x}\| = \sqrt{205}/24 = 0.5966$$

Expected measurement outcomes

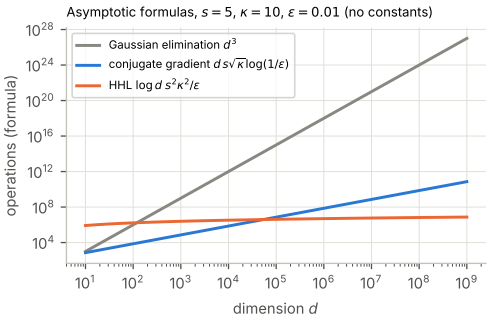
- Ancilla $= 1$ in 35.6 % of the shots (with clock 0000).
- Among those: $|00\rangle$: 76.2 %, $|01\rangle$: 6.0 %, $|10\rangle$: 14.8 %, $|11\rangle$: 3.0 % ($= |x_i|^2/\|\vec{x}\|^2$).
- Signs of x_i are *not* visible in these counts.

Where the cost of HHL comes from

Source	Cost factor	Reason
qubits for the data	$\log d$	amplitude encoding
simulating e^{iAt} (s-sparse)	$s^2\, t$ (original)	sparse Hamiltonian simulation
phase-estimation precision	$t_{\text{total}} \sim \kappa/\epsilon$	resolve $\lambda_{\min}$ to relative error ϵ
repetitions	κ^2, or κ with amplification	$P_{\text{success}} \gtrsim 1/\kappa^2$
total (HHL 2009)	$\tilde{O}(\log(d)\, s^2 \kappa^2/\epsilon)$	

- Exponential improvement in d over $O(d\, s\sqrt{\kappa})$ (conjugate gradient) — **only** if s, κ, $1/\epsilon$ are $\mathrm{polylog}(d)$.
- Doubling κ quadruples the HHL run time. Clock qubits: $n_c \approx \lceil \log_2(\kappa/\epsilon) \rceil$ (e.g. $\kappa = 100$, $\epsilon = 0.01$: 14).

Theoretical speed-up vs. practical advantage



Plots of the formulas only (constants unknown, not measurements).

- The curves cross only at large d; real constants (error correction, Hamiltonian simulation depth) push the crossover further out.
- The comparison is unfair to classical methods if they must output $\vec{x}$ and HHL does not.
- Fair question: "estimate $\langle x|M|x\rangle$". Then classical methods still need $\sim d$, HHL $\sim \log d$.
- Sparse, well-conditioned systems with efficient $|b\rangle$ are **BQP-complete**: there is a genuine quantum advantage in principle.

The fine print (Aaronson, 2015)

1. **Input**: $|b\rangle$ must be prepared quickly — arbitrary data costs $O(d)$ (or QRAM).
2. **Matrix**: e^{iAt} must be simulable — sparse or structured A.
3. **Conditioning**: κ must be small — many PDE discretisations have $\kappa \propto d^{2/D}$.
4. **Output**: only a property of $|x\rangle$ — reading $\vec{x}$ costs $\Omega(d)$.

Dequantisation

If A has **low rank** and the classical computer gets the same kind of access (sampling rows with probability $\propto$ norm^2), classical algorithms also run in $\mathrm{polylog}(d)$ (Tang 2019, Chia et al. 2020). The quantum advantage is then at most polynomial.

Advantage is expected for sparse, *full-rank*, well-conditioned problems.

`HHL.ipynb` at a glance

- Generated by `dev/notebook_builders/build_hhl.py`; 10 sections like every notebook of the suite.
- **Inputs** (Section 3): `MATRIX_A`, `VECTOR_B`, `NUM_CLOCK_QUBITS = 4`, `EVOLUTION_TIME = "auto"`, `SHOTS = 20000`.
- **Validation**: square, sizes match, $\vec{b} \neq 0$, A non-singular; computes `np.linalg.solve` as reference.
- **Preparation**: Hermitian embedding and padding; κ, t, C, ideal clock values, qubit count.
- **Backends**: `statevector` or `density_matrix` on CPU (`qpp-cpu`, `density-matrix-cpu`) or GPU (`nvidia`). Hardware targets are **refused**.

What is *not* implemented

- Hamiltonian simulation: e^{iAt} is the *exact* `scipy.linalg.expm`, registered as a custom gate.
- Efficient state preparation: $|b\rangle$ is loaded by exact amplitude initialisation.
- Amplitude amplification, noise models for HHL, hardware runs.

These keep the demonstration exact on small systems; they are the parts a real device would need.

Code walkthrough 1: preparation and phase estimation

```
U_step = expm(1j * A_work * t_evolution) # U = exp(i A t), exact (unitary)
kernel = cudaq.make_kernel()
initial = np.kron(np.kron(np.eye(2 ** nc)[0], b_work / b_norm), np.array([1.0, 0.0])) # |0>_clock |b> |0>_anc
reg = kernel.qalloc(bit_reverse_order(initial.astype(complex), n_tot))
clock = [reg[i] for i in range(nc)]; system = [reg[nc + i] for i in range(n_b)]; ancilla = reg[n_tot - 1]
# ---- 1) quantum phase estimation ----
for q in clock:
    kernel.h(q) # clock superposition
for j in range(nc):
    name = f"hhl_{gate_uid}_{nc}_u{j}"
    cudaq.register_operation(name, np.linalg.matrix_power(U, 2 ** j)) # U^(2^j) = exp(i A t 2^j)
    getattr(kernel, name)(clock[nc - 1 - j], *system) # controlled by the clock bit of weight 2^j
for j in range(nc // 2): # inverse QFT on the clock register
    kernel.swap(clock[j], clock[nc - 1 - j])
for jj in range(nc):
    j = nc - 1 - jj
    for kk in range(nc - 1 - j):
        k = nc - 1 - kk
        kernel.cr1(-np.pi / 2 ** (k - j), clock[k], clock[j])
    kernel.h(clock[j])
```

Excerpt of `build_hhl_kernel` (Section 7a). `bit_reverse_order` converts to CUDA-Q's qubit ordering; the custom gate gets a control qubit simply by being called with one extra leading qubit.

Code walkthrough 2: inversion, uncomputation, read-out

```
# ---- 2) eigenvalue inversion: R_Y(2 arcsin(C / lambda_k)) on the ancilla for every clock value k ---
for k_val in range(1, 2 ** nc):
    signed = k_val - 2 ** nc if k_val >= 2 ** (nc - 1) else k_val # two's complement: negative eigenvalues
    lam_k = 2 * np.pi * signed / (2 ** nc * t_used)
    theta = 2 * np.arcsin(np.clip(c_used / lam_k, -1.0, 1.0))
    zero_bits = [i for i in range(nc) if not (k_val >> (nc - 1 - i)) & 1] # clock[0] is the most significant bit
    for i in zero_bits: kernel.x(clock[i]) # select |k> with X "polarity" gates
    kernel.cry(theta, clock, ancilla) # multi-controlled rotation
    for i in zero_bits: kernel.x(clock[i])
# ---- 3) uncompute phase estimation (QFT, inverse controlled powers, H) ----
...
# ---- read-out (execution cell) ----
amplitudes = postselected_amplitudes(state_vector) # clock = 0, ancilla = 1
x_quantum_full = amplitudes * b_norm / C_constant # norm recovery
for bits, c in counts.items(): # shots: bitstring = clock | system | ancilla
    if bits[:n_c] == "0" * n_c and bits[-1] == "1": ... # post-selection
```

Qubit layout and circuit resources

Register (one `qalloc`): qubits 0–3 clock, 4–5 system, 6 ancilla. Measured bit strings read clock | system | ancilla.

Recorded statistics of the full 7-qubit circuit (amplitude encoding not counted):

- 111 gates, depth 67, 39 multi-qubit gates
- `h`: 16, `cr1`: 12, `swap`: 4, `x`: 56
- `ccccry`: 15 (inversion)
- custom controlled $e^{\pm iAt2^j}$: 8

Scaling with the clock

- rotations $2^{n_c} - 1$; controlled evolutions $2n_c$
- $n_c = 5$: 31 rotations, 10 evolutions, 8 qubits
- QFT parts: $O(n_c^2)$ — negligible

Memory

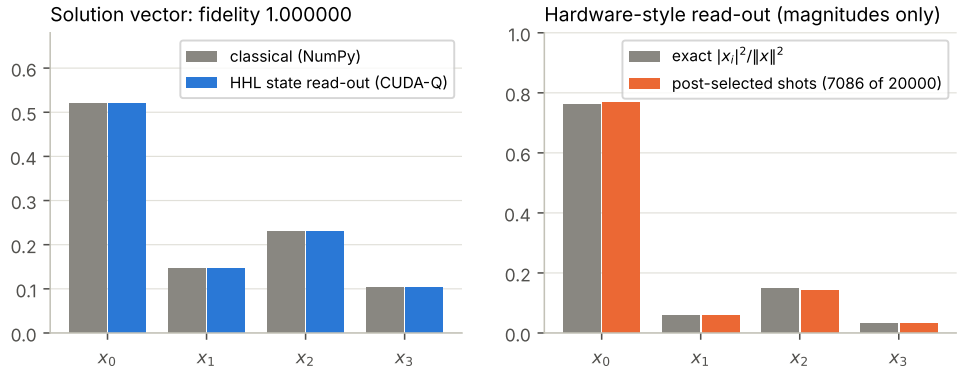
16×2^n bytes (fp64): 7 qubits = 2 KB. A $2^{20} \times 2^{20}$ system with $n_c = 10$: 31 qubits = 34.4 GB — and each dense e^{iAt2^j} gate would need 17.6 TB.

CPU and GPU simulation

BACKEND + method	target	status
cpu + statevector	qpp-cpu	run (recorded)
cpu + density_matrix	density-matrix-cpu	available
gpu + statevector	nvidia (fp32/fp64)	not executed
multi-gpu	nvidia mgpu	not executed
hardware	—	refused

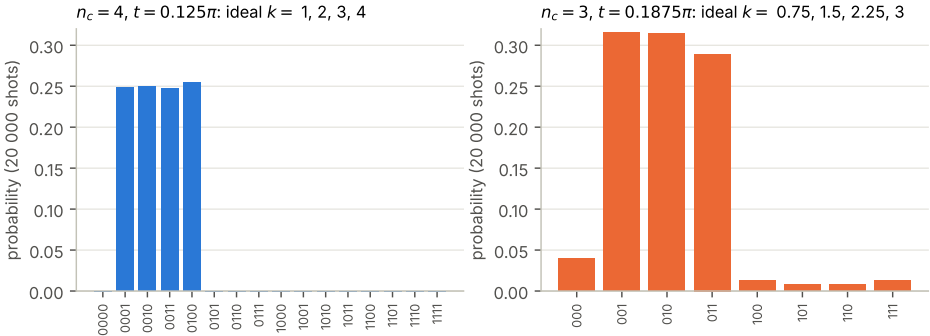
- "auto" picks a GPU when CUDA-Q sees one, else the CPU.
- The test machine had no GPU: GPU and multi-GPU paths follow the CUDA-Q 0.16 API but were **not executed**.
- Timing of the recorded run: construction 0.006 s, compilation 0.05 s, state run 0.002 s, sampling 0.08 s.

Results of the default run



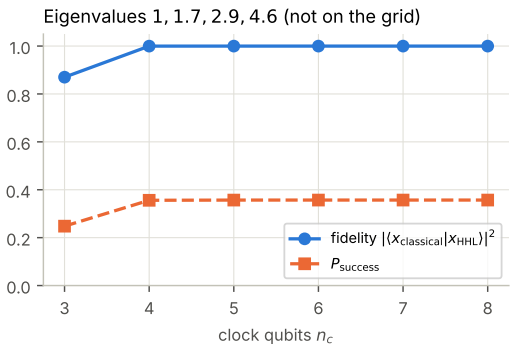
Fidelity 1.0000000000, relative error $\approx$ 1.4e-15 (state read-out, $\vec{x} = \|\vec{b}\|\vec{a}/C$). Success probability 0.355903; 7086 of 20 000 shots accepted (0.3543, standard error 0.0034). Re-computed for these slides with CUDA-Q 0.16 on `qpp-cpu`; identical to the outputs stored in `HHL.ipynb`.

What the clock register sees



Clock measured right after phase estimation (CUDA-Q, 20 000 shots). **Left**: $n_c = 4$, $t = \pi/8$ — the four eigenvalues land exactly on $k = 1, 2, 3, 4$ with probability $|\beta_j|^2 = \frac{1}{4}$ each. **Right**: $n_c = 3$ — $\kappa = 4$ exceeds the 3 positive levels, "auto" rescales $t = 3\pi/16$, the clock values $0.75, 1.5, 2.25, 3$ are rounded and the probability spreads.

Precision: clock qubits vs. fidelity



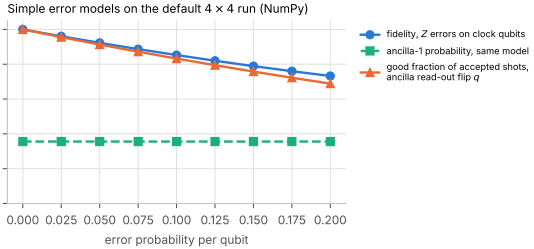
- Random 4×4 matrix with eigenvalues $1, 1.7, 2.9, 4.6$ (not multiples of C), $\vec{b} = (1, 1, 1, 1)$; notebook circuit in CUDA-Q.
- $n_c = 3$: fidelity 0.870; $n_c = 8$: 0.9998.
- Rounded eigenvalues apply slightly wrong $1/\tilde{\lambda}$; more clock qubits reduce the rounding.
- The default example is exact only because its eigenvalues were *chosen* to be integers.

Try it yourself: `HHL.ipynb`

Setting in Section 3 (`MATRIX_A`, `VECTOR_B`, ...)	**What you will see**
defaults	4×4, 7 qubits, fidelity 1, $P = 0.3559$
`[[1, -1/3], [-1/3, 1]]`, `[1, 0]`	2×2, eigenvalues $\frac{2}{3}, \frac{4}{3}$, $\vec{x} = (\frac{9}{8}, \frac{3}{8})$
`NUM_CLOCK_QUBITS = 3`	t rescaled, rounding, fidelity < 1
`"random"`, `RANDOM_CONDITION_NUMBER = 20`	P_{success} drops, more clock qubits needed
`[[1, 2], [0, 1]]`, `[1, 1]`	automatic Hermitian embedding (+1 qubit)
`NUM_CLOCK_QUBITS = 8`	11 qubits, 255 inversion rotations

Run: `jupyter lab HHL.ipynb`, edit only Section 3, *Restart & Run All*. Reports go to `exports/hhl/`.

Noise and error analysis



- **Clock phase errors** (Z with probability p per clock qubit, NumPy model): wrong eigenvalue labels, broken uncomputation. Fidelity 0.853 at $p = 0.1$.
- **Ancilla read-out flips**: failed runs are accepted; the good fraction of accepted shots falls fast because P_{success} is only 0.36.
- **Algorithmic errors**: QPE rounding, Trotter error, finite C.
- **Statistical**: $\sigma = \sqrt{p(1-p)/N_{\text{acc}}}$.

These are idealised models for illustration — the notebook itself contains no HHL noise simulation.

Limitations

- **Depth**: controlled e^{iAt2^j} with $2^{n_c} \sim \kappa/\epsilon$ total time, plus the inversion — far beyond today's noisy devices except for toy problems.
- **Fault tolerance** needed for meaningful sizes.
- **Input/output bottlenecks** (state preparation, read-out).
- **κ-dependence**: real problems are often ill conditioned; preconditioning helps but costs.
- **Dequantisation** for low-rank problems.

About this implementation

Exact e^{iAt} matrices, exact amplitude loading and the 2^{n_c}-rotation look-up table make the notebook an *exact educational simulation*. It shows that the mathematics works; it does not show a speed-up, and it cannot run on hardware.

Applications

- **Differential equations**: discretised PDEs/ODEs (heat, wave, Poisson) as sparse systems; output = an averaged quantity of the solution.
- **Electromagnetic scattering** cross-sections (Clader, Jacobs, Sprouse 2013).
- **Machine learning**: least squares, quantum SVM, Gaussian processes (subject to dequantisation caveats).
- **Network/graph problems**: effective resistance, hitting times.
- Subroutine inside **quantum chemistry** and **optimisation** pipelines.

Recipe for a good application

1. sparse, structured A
2. $\kappa = \mathrm{polylog}(d)$ (or good preconditioner)
3. $|b\rangle$ from a formula, not from a data file
4. answer = one number, e.g. $\langle x|M|x\rangle$

Advanced quantum linear-system algorithms

Algorithm	**Key idea**	**Query complexity**
HHL (2009)	QPE + controlled rotation	$\tilde{O}(\kappa^2/\epsilon)$
Ambainis (2012)	variable-time amplification	$\tilde{O}(\kappa \log^3 \kappa/\epsilon^3)$
Childs–Kothari–Somma (2017)	LCU series for $1/x$	$\tilde{O}(\kappa \operatorname{polylog}(\kappa/\epsilon))$
QSVT (Gilyén et al. 2019)	polynomial of singular values	$\tilde{O}(\kappa \operatorname{polylog}(\kappa/\epsilon))$
Adiabatic (Subaşı et al. 2019)	randomised adiabatic	$O(\kappa \log \kappa/\epsilon)$
Time-optimal adiabatic (An–Lin 2022)	optimised schedule	$\tilde{O}(\kappa \operatorname{polylog}(\kappa/\epsilon))$
Discrete adiabatic (Costa et al. 2022)	walk + adiabatic schedule	$O(\kappa \log(1/\epsilon))$, optimal

Also: variational linear solvers (VQLS) for near-term devices — heuristic, no proven speed-up.
All share HHL's input/output caveats.

Exercises

1. Find the eigenvalues and κ of $\left(\begin{smallmatrix} \frac{5}{2} & \frac{2}{2} \end{smallmatrix}\right)$.
2. $\vec{b} = (3, 0, 4, 0)$ is amplitude-encoded. What is $P(|10\rangle)$?
3. With $n_c = 4$ and $t = \pi/8$: which bit string encodes $\lambda = 3$? Which λ does 1110 encode (two's complement)?
4. With $C = 1$, what ancilla amplitude and success probability does the component with $\lambda = 4$ get?
5. $A = \mathrm{diag}(1, 2)$, $|b\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, $C = 1$. Compute P_{success} and $|x\rangle$.
6. How many qubits does HHL need for an 8×8 Hermitian matrix with $n_c = 5$?
7. Why must the clock register be uncomputed instead of measured?
8. For the default run, derive $P_{\text{success}} = 205/576$ and $\|\vec{x}\|$.

Answers

1. $\lambda^2 - 7\lambda + 6 = 0$: $\lambda = 1, 6$; $\kappa = 6$.
2. $|b\rangle = 0.6\,|00\rangle + 0.8\,|10\rangle$: $P = 0.64$.
3. $k = 3 \to 0011$; $1110 = 14 \to 14 - 16 = -2$, so $\lambda = -2$.
4. Amplitude $C/\lambda = \frac{1}{4}$, probability $\frac{1}{16}$.
5. $P = \frac{1}{2}(1 + \frac{1}{4}) = \frac{5}{8}$; branch $\propto |0\rangle + \frac{1}{2}\,|1\rangle$, $|x\rangle = \frac{1}{\sqrt{5}}(2\,|0\rangle + |1\rangle)$.
6. $3 + 5 + 1 = 9$.
7. Measuring collapses to one eigenvector; leaving it entangled gives a mixed state. Only uncomputation lets the components interfere into $|x\rangle$.
8. $P = \frac{1}{4}(1 + \frac{1}{4} + \frac{1}{9} + \frac{1}{16}) = \frac{205}{576}$; $\|\vec{x}\| = \frac{\|\vec{b}\|}{C}\sqrt{P} = \frac{\sqrt{205}}{24} \approx 0.597$.

More practice: the 80-question test `assessments/HHL_Question_Paper.pdf` with full solutions in `HHL_Answer_Book.pdf`.

Summary

1. $A^{-1}\vec{b} = \sum_j (\beta_j/\lambda_j)\,|u_j\rangle$: invert every eigenvalue.
2. **QPE** with e^{iAt} writes λ_j into a clock register.
3. A **controlled** R_Y puts C/λ_j on an ancilla.
4. **Uncomputation** erases the clock; **post-selection** on ancilla $= 1$ leaves $|x\rangle$.
5. $P_{\text{success}} = C^2 \|A^{-1}\,|b\rangle\|^2 \gtrsim 1/\kappa^2$.
6. Cost $\tilde{O}(\log(d) s^2 \kappa^2/\epsilon)$ — under strong assumptions.

Take home

HHL turns linear algebra into *state preparation*. Its power is real but conditional: sparse, well-conditioned A, cheap $|b\rangle$, and a question about $\vec{x}$ that one number can answer.

In CUDA-Q

The notebook reproduces $\vec{x} = \frac{1}{48}(25, 7, 11, 5)$ with fidelity 1 on a CPU simulator — an exact demonstration, not a speed-up.